



Shell脚本实践

誉天教育

- Shell脚本编写规范
- Shell的四种执行方式
- 位置化变量和预定义变量
- 条件测试语句
- if、case判断语句
- for、while、until循环



- Shell脚本是实现自动化运维必备的重要工具，能够批量重复话操作，减少Linux管理员工作量，简化日常维护工作
- 基于Linux命令，上手操作迅速
- 解释性语言，简单易学
- 自动化运维

如何去学习好shell编程

- 熟悉vim编辑器
- 熟悉使用Linux常用命令
- 熟练文本处理工具（grep、awk、sed）
- 熟悉常用服务的部署、优化、日志及排错

1. 开头指定shell解释器
\$ #!/bin/bash 或 #!/bin/sh
2. 脚本开头指定作者、版权等个人信息
\$ #Author:作者
\$ #Mail:联系方式
\$ #Date:日期
\$ #Describe:描述
\$ #Warning:警告信息
\$ #Version:版本
\$ #Modify:记录修改信息，方便查看和维护
3. 脚本后缀以 .sh 结尾
4. 代码缩进
5. 尽量不使用中文注释

```
[root@rhel9 ~]# vim test.sh
#!/bin/bash

#Author:yutianedu
#Mail:www.yutianedu.com
#Date:2001
#Describe:Wuhan
#Warning:Wuhan
```

使用 .vimrc 文件，能够快速的生成开头的注释信息

```
[root@ scripts]# cat ~/.vimrc
autocmd BufNewFile *.py,*.cc,*.sh,*.java exec ":call SetTitle()"
func SetTitle()
if expand("%:e") == 'sh'
call setline(1,"#!/bin/bash")
call setline(2,"#####")
call setline(3, "# File Name: ".expand("%"))
call setline(4, "# Version: V1.0")
call setline(5, "# Author: sunyaping")
call setline(6, "# Email: sunyaping@yutianedu.com")
call setline(7, "# Created Time : ".strftime("%F %T"))
call setline(8, "# Description:")
call setline(9,"#####")
call setline(10, "")
endif
endfunc
```

- 注释的作用：描述此代码作用，不参与脚本的执行
1. 单行注释：以 **#** 开头，到行尾结束
 2. 多行注释：以 **<<EOF** 开头，到**EOF**结束(EOF可以自定义)

```
#!/bin/bash

#Author:yutianedu
#Mail:www.yutianedu.com
#Date:2001
#Describe:Wuhan
#Warning:Wuhan

<<EOF
Author:yutianedu
Mail:www.yutianedu.com
Date:2001
Describe:Wuhan
Warning:Wuhan
EOF
```

- ① 以相对路径的方式执行
 - ② 以绝对路径的方式执行
 - ③ source 脚本
 - ④ bash 脚本
- 区别：
 1. 相对路径和绝对路径执行脚本都需要 **x** 权限
 2. bash 执行脚本不需要指明 shell 解释器
 3. source 执行脚本是在当前shell中执行，其他都是子shell中执行

示例：使用不同的方式执行下方脚本，查看不同执行方式的区别

```
[root@rhel9 ~]# cat test.sh
#!/bin/bash
redhat=999
echo yutian
```


- 位置变量是往脚本传递参数或数据的，变量名不能修改

位置参数变量	作用
\$n	\$0表示脚本名，\$1-\$9表示第一个参数到第9个参数，9之后的参数用\${n}表示

```
[root@rhel9 ~]# cat test.sh
#!/bin/bash
echo "脚本名称: $0"
echo "第一个参数: $1"
echo "第二个参数: $2"
[root@rhel9 ~]# bash test.sh 1 2
脚本名称: test.sh
第一个参数: 1
第二个参数: 2
```

- 是 Bash 中已经定义好的变量，变量名不能自定义，变量作用也是固定的

预定义变量	作用
\$?	上一个命令的返回值，0成功，1失败
\$\$	当前进程的PID
\$_	后台任务的最后一个ID
\$*	表示命令行的所有参数，\$表示把所有参数看成一个整体
\$@	表示命令行的所有参数，@表示每个参数都是个体
\$#	表示传入参数的个数

- 语法：read [选项] [变量名]
- 选项：

- p 提示信息

- t 等待的秒数

- s 隐藏信息输入

read命令行中也可以不指定变量，如果不指定变量，那么read命令会将接收到的数据放置在环境变量REPLY中

- 示例：编写user.sh脚本，创建用户user1，输入密码为隐藏输入

```
#!/bin/bash
read -p "请输入创建的用户名：" user_name
read -s -p "请输入用户的密码：" user_passwd

useradd $user_name
echo $user_passwd|passwd --stdin $user_name
```

- 程序执行后会携带一个退出码：
 - (1) 0代表成功，1-255代表失败
 - (2) Shell程序的退出码储存在系统变量\$?中
 - (3) exit num可以中断程序并设置退出码为num
- 举例：

```
$ ping -c1 baidu.com &> /dev/null
$ echo $?
0
```

- 条件是否执行根据退出码决定

逻辑运算符	作用
&&	逻辑 “ 与 ”；前面的命令判断结果为true，执行后面的命令
	逻辑 “ 或 ”；前面的命令判断结果为false，执行后面的命令
!	逻辑 “ 非 ”；对命令语句的判断结果取反

➤ 示例：

```
# ls /root && cd /tmp
# ls /root || cd /tmp
# [root@rhel9 ~]# ping -c 1 -W1 baidu.com \
> && echo success_ping \
> || echo error
```

- 多条命令的集合，用 “{}” 和 “()” 将多条命令括起来，使其在语法上成为一条命令

指令	示例
()	# (pwd;cd /tmp;pwd)
{ }	# { 空格 pwd;cd /tmp;pwd; ; }

➤ 区别：

()在子shell中执行；{ }在当前shell中执行

()开头不需要空格，最后一个命令不用分号；{ }开头必须有空格，结尾必须有分号

案例：

```
[root@localhost scripts]# (pwd;cd /tmp;pwd)
```

```
[root@localhost scripts]# { pwd;cd /tmp;pwd;}
```

() 子shell中执行，执行完毕，当前路径不变

- 使用测试语句来测试指定的条件表达式的真或者假

当条件为真时，返回值为0；

当条件为假时，返回值为非0；

- 格式：

test 条件表达式；

[条件表达式]；

[[条件表达式]]；（支持正则和通配符）

- 检测文件的属性

操作符	说明	举例
-b file	检测文件是否是块设备，是则返回True	[-b \$file] 返回 True
-f file	检测文件是否是普通文件，是则返回True	[-f \$file] 返回 True
-d file	检测文件是否是目录文件，是则返回True	[-d \$file] 返回 True
-k file	检测文件是否设置sgid，是则返回True	[-k \$file] 返回 True
-u file	检测文件是否设置suid，是则返回True	[-u \$file] 返回 True
-r file	检测文件是否设置r权限，是则返回True	[-r \$file] 返回 True
-w file	检测文件是否设置w权限，是则返回True	[-w \$file] 返回 True
-x file	检测文件是否设置x权限，是则返回True	[-x \$file] 返回 True
-p file	检测文件是否是具名管道，是则返回True	[-p \$file] 返回 True
-s file	检测文件是否为空，是则返回True	[-s \$file] 返回 True
-e file	检测文件是否存在，是则返回True	[-e \$file] 返回 True

- 比较数值的大小

操作符	说明	举例
-eq	检测两个数是否相等，相等返回true	[\$a -eq \$b] 返回 True
-ne	检测两个数是否相等，不相等返回true	[\$a -ne \$b] 返回 True
-gt	检测左边的数是否大于右边，是则返回true	[\$a -gt \$b] 返回 True
-lt	检测左边的数是否小于右边，是则返回true	[\$a -lt \$b] 返回 True
-ge	检测左边的数是否大于等于右边，是则返回true	[\$a -ge \$b] 返回 True
-le	检测左边的数是否小于等于右边，是则返回true	[\$a -le \$b] 返回 True

判断变量是不是数字：

```
[root@localhost ~]# num10=123
```

```
[root@localhost ~]# num20=ssss1114ss
```

```
[root@localhost ~]# [[ "$num10" =~ ^[0-9]+$ ]];echo $?
```

操作符	说明	举例
!	非运算，表达式为true则返回false，否则返回true	[!false] 返回 True
-o	或运算，有一个表达式为true，则返回true	[\$a -lt 20 -o \$b -gt 100] 返回 True
-a	与运算，两个表达式都为true，才返回true	[\$a -lt 20 -a \$b -gt 100] 返回 True

示例：

```
[root@localhost ~]# [ -f /etc/hosts -a -f /etc/services ] && echo yes || echo no
```

```
[root@localhost ~]# [[ -f /etc/hosts && -f /etc/services ]] && echo yes || echo no
```

操作符	说明	举例
=	比较字符串内容是否相同，相同返回true	[\$a = \$b] 返回 True
!=	比较字符串内容是否相同，不相同返回true	[\$a != \$b] 返回 True
-z	判断字符串内容是否为空，为空返回true	[-z \$a] 返回 True

条件判断-if语句

- If单分支结构

- 格式1

If <条件表达式>

then

命令语句

fi

- 格式2

If <条件表达式> ; then

命令语句

fi

条件判断-if语句

- If双分支结构
 - 格式

```
If <条件表达式>
then
    命令语句
else
    命令语句
fi
```

示例：

```
if [ -d /root1 ]
then
    echo success
else
    echo erro
fi
```

条件判断-if语句

- If多分支结构
 - 格式

```
If <条件表达式>
then
    命令语句
elif
then
    命令语句
...
...
else
    命令语句
fi
```

示例 1：检查主机是否存活，并输出结果

```
ping -c 2 -W 1 192.168.150.16 &> /dev/null
```

示例 2：根据用户输入成绩，判断优良中差。

85-100 优秀--A 70-84 良好--B 60-69 合格--C 60分以下不合格--D

```
read -p "Please enter your score (0-100): " grade
```

```
if [ $grade -ge 85 ]; then
```

```
echo "$grade, A"
```

```
elif [ $grade -ge 70 ]; then
```

```
echo "$grade, B"
```

```
elif [ $grade -ge 60 ]; then
```

```
echo "$grade, C"
```

```
else
```

```
echo "$grade, D"
```

```
fi
```

条件判断-case语句

- Case语法格式

case 变量值 in

值1)

命令语句

::

值2)

命令语句

::

.....

*)

默认命令语句

esac

示例-case语句

示例：将判断分数范围用case语句实现

```
read -p "Please enter your score (0-100): " grade
case "$grade" in
    8[5-9]|9[0-9]|100)
        echo "$grade, A"
        ;;
    7[0-9]|8[0-4])
        echo "$grade, B"
        ;;
    6[0-9])
        echo "$grade, C"
        ;;
    *)
        echo "$grade, D"
esac
```

- 格式
for 变量名 in {取值的列表}
do
 命令语句
done

方法1：直接列出元素方法

```
for i in 1 2 3 4 5  
do  
    echo $i  
done
```

方法2：使用大括号

```
for i in {1..5}  
do echo $i  
done
```

方法3：使用seq

```
for i in $(seq 1 5)  
do  
    echo $i  
done
```

方法4：使用命令的结果

```
for i in $(ls *.sh)  
do  
    echo $i  
done
```

示例1-for循环

- 使用for循环语句创建多个用户，创建的用户都放在文件中

```
[root@rhel9 opt]# cat user_list
```

```
zhangsan
```

```
bob
```

```
harry
```

```
lisi
```

```
wangwu
```

```
#!/bin/bash
```

```
#输入用户的秘密
```

```
echo -n "Please input password: "
```

```
read -s passwd
```

```
#循环用户列表里面的用户，如果存在则提示用户已经存在，不存在则创建用户并且设置密码
```

```
for username in $(cat /opt/user_list)
```

```
do
```

```
    id $username &> /dev/null
```

```
if [ $? -eq 0 ]
```

```
then
```

```
    echo "$username is exist!"
```

```
else
```

```
    useradd $username
```

```
    echo "$passwd" | passwd --stdin $username &> /dev/null
```

```
    echo "$username" success!
```

```
fi
```

```
done
```

- 示例：编写一个 Shell 程序，实现判断当前网络（假定为 192.168.1.0/24，根据实际情况实现）里，当前在线用户的IP有哪些？

```
for ((i=1;i<=254;i++))
do
    if ping -c 2 -W 1 192.168.1.$i &>/dev/null
    then
        echo "192.168.1.$i is up..."
    else
        echo "192.168.1.$i is down..."
    fi
done
```

- 在while中只要条件成立就会一直循环，当条件不成立时就会退出
- 语法格式：

While <条件表达式>

do

statement1

statement2

.....

done

- while循环读取文件
- 方法一：采用exec读取文件，然后进入while循环处理

```
exec < File ; while read line
do
    statement1
done
```
- 方法二：使用cat读文件，然后通过管道进入while循环处理

```
cat File | while read line
do
    statement1
done
```
- 方法三：通过在while循环结尾，使用输入重定向方式

```
while read line
do
    statement1
done < File
```



循环语句-while



Red Hat



- 示例：while读取文件工作过程中遇到要从一个ip列表中获取ip port，然后ssh ip 到目标机器进行特定的操作

```
# cat iplist
```

```
192.168.147.101 5225
```

```
192.168.147.102 2234
```

```
192.168.147.103 4922
```

```
# cat while-read.sh
```

```
#!/bin/bash
```

```
while read line
```

```
do
```

```
IP=$(echo $line |awk '{print $1}')
```

```
PORT=$(echo $line |awk '{print $2}')
```

```
echo "IP: $IP, PORT: $PORT"
```

```
done <ipilist
```



循环语句-until



Red Hat



誉天教育
YU TIAN EDU

- until用法与while用法类似，唯一的区别是，until循环是在条件表达式不成立的时候，进入do循环，如果成立则终止循环

- 语法格式：

until <条件表达式>

do

命令语句

done



break和continue

- 在Shell中的循环结构中，还有2个语句非常有用，即break和continue语句。前者用于立即从循环中退出；而后者则用来跳过循环体中的某些语句，继续执行下一次循环。
- break语句的作用是立即跳出某个循环结构。break语句可以用在for、while或者until等循环语句的循环体中。
- continue语句则比较有趣，它的作用不是退出循环体。而是跳过当前循环体中该语句后面的语句，重新从循环语句开始的位置执行。



示例-break和continue



Red Hat



誉天教育
YU TIAN EDU

- 示例 1

```
# cat for-break.sh
#!/bin/sh
for i in `seq 10`
do
    if [ $i -eq 4 ]
    then
        break
    fi
    echo $i
done
```

- 示例 2

```
# cat for-continue.sh
#!/bin/sh
for i in `seq 10`
do
    if [ $i -eq 4 ]
    then
        continue
    fi
    echo $i
done
```

- ✓ Shell脚本编写规范
- ✓ Shell的四种执行方式
- ✓ 位置化变量和预定义变量
- ✓ 条件测试语句
- ✓ if、case判断语句
- ✓ for、while、until循环

誉天教育
YU TIAN EDU



Red Hat
Enterprise
Linux

